

Hashing

- Efficient way to implement dictionaries
- Efficiently perform insertion, search and deletion
- Hashing is based on the idea of distribution keys among a one-dimensional array $H[0 \dots m-1]$.
- This array is called a hash table.
- The distribution is done by computing, for each of the keys, the value of some predefined function h .
- This function h is called the hash function.
- The hash function assigns an integer between 1 and $m - 1$ to a key.
- This assigned integer is called the hash address.
- For example, using a table size $m=10$ and a hash function $h(\text{key}) = \text{length}(\text{key}) \bmod 10$, the key "apple" (length 5) is assigned a hash address of 5.

Hash function needs to satisfy the following somewhat conflicting requirements:

- Hash table size should not be excessively large compared to number of keys, but should be sufficient to not jeopardize the implementation's time efficiency
- Needs to distribute keys among the cells of the hash table as evenly as possible
- Has to be easy to compute

Collision

- If we choose a hash table's size smaller than the number of keys n , we will get collisions
- It is the situation where 2 or more keys being hashed into the same cell of the hash table
- For example, using the length hash function and $m=10$, both the keys "apple" and "grape" (both length 5) would hash to the same index (5), causing a collision.
- Collisions should be expected even if m is considerably larger than n .
- Worse case is all keys could be hashed into the same cell of the hash table
- With appropriately chosen hash table size and good hash function, this situation happens very rarely

Types of hashing:

1. Open Hashing (Separate Chaining)
2. Closed Hashing (Open Addressing)

Open Hashing (Separate Chaining):

- Keys are stored in linked lists attached to cells of a hash table

- Each list contains all the keys hashed to its cell
- For instance, if "apple" is already stored at index 5, when "grape" also hashes to index 5, "grape" is appended to the linked list extending from cell H[5].
- Efficiency of searching depends on the lengths of the linked lists, which in turn depends on the dictionary and table sizes, as well as quality of the hash function.
- If the hash function distributes n keys among m cells of the hash table about evenly, each list will be about n/m keys long.
- The ratio $\alpha = n/m$, called the load factor of the hash table plays an important role in the efficiency of hashing
- The average number of pointers inspected for successful searches is $s = 1 + \alpha/2$ and for unsuccessful searches is $U = \alpha$, assuming a random element search and uniform key distribution.
- The search results are almost identical to searching sequentially in a linked list, what we have gained by hashing is a reduction in average list size by a factor of m , which is the size of the hash table
- We don't want the load factor to be too far from 1
 - Having it too small would imply a lot of empty lists and hence inefficient use of space
 - Having it too large would mean longer linked lists and hence longer search times.
- Insertions are normally done at the end of a list
- Deletion is performed by searching for a key to be deleted and removing it from its list
- Hence efficiency of these operations is identical to that of searching
- The average case of all is $\Theta(1)$ if the number of keys n is about equal to the hash table size m

Closed hashing (Open Addressing)

- All keys are stored in the hash table itself without the use of linked lists
- Which means table size m must be at least as large as number of keys n
- Different strategies are employed for collision resolution

Linear Probing

- The simplest collision resolution method
- Checks the cell following the one where the collision occurs
- If that cell is empty, the new key is installed there, if the next cell is already occupied, the availability of that cell's immediate successor is checked, and so on
- If the end of the hash table is reached, it wraps to the beginning of the table, i.e., it is treated as a circular array

Searching

- To search for a given key K , we start by computing $h(K)$, where h is the hash function used in the table construction

- If the cell $h(K)$ is empty, the search is unsuccessful
- If the cell is not empty, we must compare K to the cell's occupant
 - If they are equal, we have found a matching key
 - If they are not, we compare k with a key in the next cell, and continue in this manner until we encounter either a matching key or an empty cell.

Deletion

- If we simply delete the key from the last state of the hash table, we will be unable to find the key after it.
- If we try to search for a key that appears after this, the algorithm will report an unsuccessful search
- The solution is to use lazy deletion, i.e., mark the previously occupied locations by a special symbol to distinguish them from locations that have not been occupied
- $S = \frac{1}{2} (1 + 1/(1-\alpha))$ for successful searches
- $U = \frac{1}{2} (1 + 1/(1-\alpha)^2)$ for unsuccessful searches
- The accuracy of these approximations increases with larger sizes of hash table

Clustering

- As the hash table gets closer to being full, the performance of linear probing deteriorates because of a phenomenon called clustering
- Cluster in linear probing is a sequence of contiguously occupied cells (with possible wrapping)
- Clusters are bad in hashing because they make the dictionary operations less efficient.
- As clusters become larger, the probability that a new element will be attached to a cluster increases
- Large clusters are more likely to merge together when a new key is added, worsening the clustering effect.

Double Hashing

- Under this, we use another hash function $s(K)$, to determine a fixed increment for the probing sequence to be used after a collision at location $l = h(K)$: $(l + s(K) \% m)$, $(l + 2s(K) \% m)$, etc
- To guarantee that every location in the table is probed by sequence, the increment $s(K)$ and the table size m must be relatively prime
- With good hashing functions both primary and secondary, double hashing is superior to linear probing
- But performance also deteriorates when the table gets close to being full

- The solution is rehearsing: the current table is scanned and all its keys are relocated into a larger table

Hashing finishers

- With hashing, the searching, insertion, and deletion can be implemented to take $\Theta(1)$ time on average but $\Theta(n)$ in the very unlikely worst case.
- Hashing does not assume the existence of key ordering and usually does not preserve it. This makes hashing less suitable for applications that need to iterate over the keys in order or require range queries such as counting the number of keys between some lower and upper bounds.

Applications of Hashing:

- **Symbol Tables in Compilers:** Used for fast lookup of variables and functions.
- **Caches:** Used by web browsers and servers to quickly retrieve recently accessed data.
- **Database Indexing:** Employed for efficient mapping of records to disk blocks (e.g., Extendible Hashing).
- **Security and Cryptography:** Used for password verification and data integrity checks.
- **AI/Game State Tracking:** To rapidly check if a certain state or position has been previously evaluated.